I. Data Lifecycle & Schema Evolution

Standalone capability brief · Suggested module: **Data Lifecycle Platform**

Purpose

Make data change, retention, portability, recovery, and migration predictable across services, databases, events, and tenant environments.

1. Scope and capability areas

Schema and migrations

Standardize migration versioning, expand-contract changes, rollback strategy, and deployment ordering.

Event and API evolution

Maintain compatibility across producers and consumers during staggered deployments.

Lifecycle and archival

Define retention, archival, deletion, and legal-hold behavior by data class.

Portability

Support controlled data export/import with integrity checks, authorization, and resumability.

Recovery

Define backup, restore, disaster-recovery, and recovery validation contracts.

Lineage and quality

Track data provenance, quality checks, reconciliation, and tenant moves across shards or regions.

2. Shared contracts and standards

- Every data asset has an owner, classification, retention policy, source of truth, and recovery expectation.
- Schema changes use compatibility-aware rollout patterns and explicit rollback limits.
- Deletion semantics distinguish soft deletion, hard deletion, derived copies, backups, and legal holds.
- Exports and imports are authenticated, auditable, integrity-checked, and bounded by resource limits.

3. Recommended implementation approach

- Use expand-contract migrations for changes that must coexist with old and new application versions.
- Version events and validate compatibility in CI.
- Automate retention and archival jobs with dry-run reporting and exception handling.
- Test restore procedures regularly; backup completion alone is not proof of recoverability.
- Build tenant migration tooling with checkpoints, reconciliation, throttling, and rollback planning.

4. Failure behavior and operational expectations

- Stop destructive migrations when preconditions or integrity checks fail.

- Partial exports/imports must be resumable or clearly marked incomplete.
- Recovery plans should define data-loss and recovery-time objectives, with escalation paths when targets are missed.

5. Suggested operational metrics

- Migration failure rate
- Restore-test success rate
- Retention-policy compliance
- Data reconciliation discrepancies
- Tenant migration duration and rollback rate

6. Readiness and acceptance checklist

- ■ Schema changes can roll out across mixed application versions.
- ■ Retention and deletion policies are enforceable and auditable.
- ■ Restore tests validate usable data and application recovery.
- ■ Tenant migrations include integrity checks and a defined recovery path.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.